

Deepfake & Manipulated Media Detection

A triage service for video, image and audio — built to be auditable, and honest about what it does not know.

Scope	Deployment	Interface	Status
MVP, timeline reopened	Docker Compose, single GPU node	Web application	Pipeline verified end-to-end

1. Problem Statement Understanding

Synthetic and manipulated media — face swaps, reenactment video, cloned voices — has become cheap to produce and fast to distribute. On social platforms the damage is done in the first hours of circulation, long before any manual verification can catch up. The task is to build a system that helps detect such content.

Reading past the surface statement, four constraints shape everything below:

1.1 The output is a triage signal, not a verdict

No detector is correct all the time, and detection accuracy degrades sharply on media the model was not trained on. A system that returns a confident "FAKE" is more dangerous than useful: a false positive on authentic footage is itself a form of misinformation, and it is the kind that gets someone accused. The honest output is a *score with a band and an explicit uncertainty region*, routed to different handling — not a binary label.

1.2 The input is hostile

Every uploaded file is attacker-controlled. It is parsed by media decoders with a long history of memory-safety bugs, and the scores themselves can be moved by adversarial perturbation of the input. Both facts have to be designed around rather than assumed away.

1.3 The media is biometric, so retention is a liability

Uploads are faces and voices. Storing them indefinitely creates a growing breach surface and a regulatory exposure that grows with the archive. Deletion must be the default and must be *verifiable*, while still keeping enough evidence that a disputed result can be re-examined.

1.4 A result nobody can reconstruct is worthless

If a decision is challenged weeks later, "the model said 84" is not defensible unless the exact weights, the aggregation method and its parameters, and the hash of the analysed bytes were all recorded at decision time. That record is a first-class deliverable, not logging.

Explicit non-goal

This system does not decide whether content is true, nor whether it should be removed. It detects signals of *synthesis and manipulation* and routes them for handling. Authenticity and truthfulness are different questions.

2. Proposed Solution

A service that accepts video, image or audio and returns a calibrated manipulation score, a routing band, and a reproducible audit record — with the underlying biometric media deleted on a timer by default.

2.1 Three pipelines, one decision path

Each modality is analysed by a pipeline suited to it, but all three converge on shared aggregation, band routing and retention logic. This keeps one place where a score becomes a decision, rather than three subtly different ones.

2.2 Score the parts, then aggregate deliberately

A model scores individual frames, faces or audio chunks. Those per-item scores are combined with a **confidence-weighted trimmed mean** — never a plain mean, which a handful of badly aligned faces can drag anywhere. Items are weighted down by detection and alignment confidence, so a frame whose face was barely found counts for less.

2.3 Refuse to guess

When face extraction returns nothing, the result is `undetermined`. It is never quietly collapsed into "authentic". When more than one face is present, each is scored and the item rolls up to **worst-case severity**, so a manipulated face in a crowd is not averaged away.

2.4 Bands, not percentages

The aggregated score maps to a band that determines handling. The bands form a **total partition** of the range, so routing has no undefined input.

SCORE	BAND	HANDLING
< 20	Likely authentic	Auto-clear, delete on schedule
20 – 40	Leaning authentic	Auto-clear, delete on schedule
40 – 60	Uncertain	Flagged for review + alert (normal urgency)
60 – 80	Leaning manipulated	Flagged + alert (low urgency); still deletes on schedule
> 80	Likely manipulated	Flagged + alert; extended retention window opens
—	Undetermined	No usable signal; flagged, never scored as real or fake

The 60–80 region gets particular care. It sits directly below the high-severity threshold, whose calibration is not trusted enough to display as a raw percentage, so it is logged and flagged rather than passed silently into normal deletion.

3. System Architecture

3.1 Detection pipelines

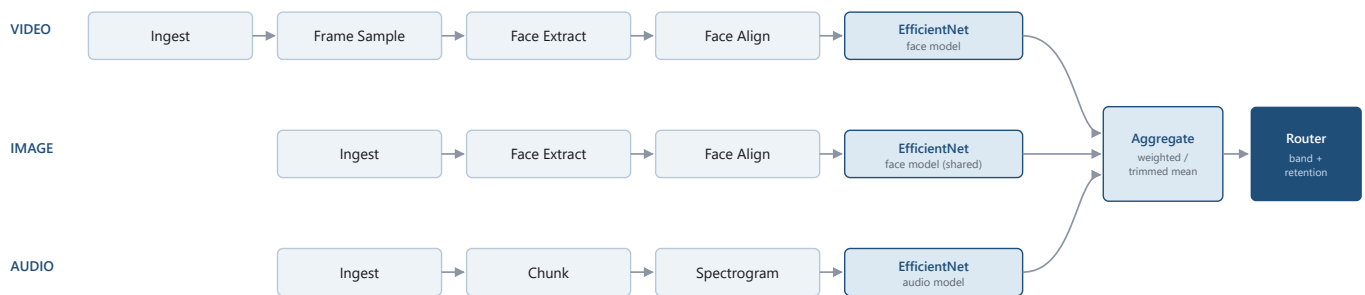


Figure 1 — Three ingest pipelines converging on shared aggregation and routing.

3.2 Service topology and trust boundary

Network layout is a security control, not organisation. The container that parses attacker-supplied media has no route off the host.

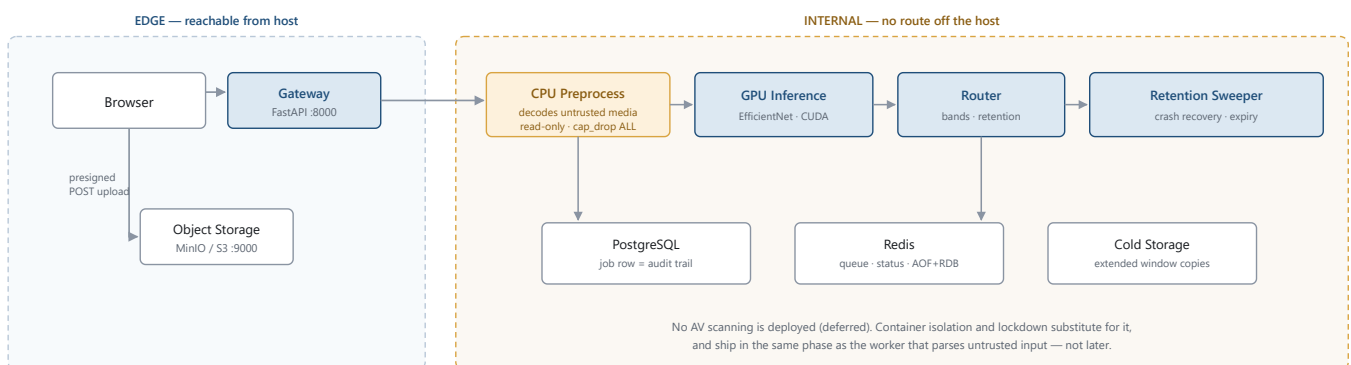


Figure 2 — Two-zone topology. Only the gateway and object storage are reachable from outside.

3.3 Request lifecycle

1. Client requests a job. The gateway rate-limits, creates the Postgres row, and returns a **presigned POST** grant.
2. Client uploads bytes *directly* to object storage — large media never transits the API.
3. Client notifies the gateway, which enqueues the job. The notify endpoint is idempotent.
4. CPU worker decodes, samples, extracts and aligns faces (or chunks audio into spectrograms).
5. GPU worker scores every item and writes per-item rows.
6. Router aggregates, assigns a band, raises flags, and triggers deletion of the working media.
7. Status is pushed over WebSocket throughout, with polling as the reconnect fallback.

4. Key Features

4.1 Verifiable deletion by default TIER 1

Raw uploads and derived face crops are TTL-deleted when inference completes. Every delete path — the completion trigger, the crash-recovery sweeper, and cold-storage expiry — checks a hold flag first. The flag column and the check that reads it shipped in the same commit as the delete path, before anything could set it true.

4.2 The job row is the audit trail TIER 1

Every result carries the SHA-256 of the analysed bytes, the `model_version_id` that produced the scores, and the exact aggregation method and parameters. Per-item scores are retained after the media is deleted — they are numbers, not biometrics — so a disputed result stays examinable without keeping anyone's face.

4.3 Upload path that cannot be used as free storage TIER 1

Uploads bypass the API by design, which means ingress rate limiting never sees those bytes. The grant is therefore a POST policy carrying a signed `content-length-range` condition, which object storage itself enforces. A presigned PUT would have signed the URL but not the body length — an unbounded write to the bucket.

4.4 Extended retention window TIER 2

A high-band result copies *the face crops that drove the score* — not the full raw source — into cold storage under a fixed 30-day timer, so a dispute has the specific evidence to re-examine while the original upload is still deleted on schedule.

This is not a legal hold

It is a fixed timer that auto-expires, and it is described that way everywhere in the code, the API responses and this document. A timer that quietly expires mid-dispute would be a liability disguised as a safeguard. Relying on it for an actual dispute needs legal sign-off that this codebase does not attempt to assert.

4.5 Isolation instead of AV scanning TIER 3 SUBSTITUTE

The preprocessing worker opens attacker-supplied media with no antivirus in front of it. It runs on an internal-only network with no route off the host, read-only root filesystem, all capabilities dropped, `no-new-privileges`, a `noexec` tmpfs, and process and memory caps.

4.6 Resilience and honest failure

- **DLQ** — bounded retries, then dead-letter with a log line, rather than silent loss or infinite retry.
- **Durable status** — Redis runs with AOF and RDB enabled, and the service refuses to start without it, so a restart cannot vaporise in-flight job state.
- **Crash recovery** — a sweeper finds completed jobs still holding bytes and finishes the deletion, subject to the same hold gate.

4.7 Advisories attached to every result

Results carry machine-readable advisories stating that scores are manipulable by adversarial perturbation, and — while the stub backend is active — that the score is a placeholder rather than a detection.

5. Technology Stack

LAYER	CHOICE	RATIONALE
API	Python 3.11, FastAPI, Uvicorn	Native async for WebSocket status push; typed request/response models.
Models	PyTorch, EfficientNet	Strong accuracy-per-FLOP; separate face and audio models, each with its own calibration.
Calibration	Temperature scaling	Cheap, monotonic, needs only a held-out set. Launch snapshot only.
Database	PostgreSQL 16 (psycopg3)	Transactional audit trail; JSONB for aggregation params; partial indexes drive the sweepers.
Queue & status	Redis 7 (AOF + RDB)	One dependency for both work queue and live status. Persistence is mandatory, not default.
Object storage	MinIO locally, S3 deployed (boto3)	Identical presigned-upload semantics in both; MinIO disappears from compose in a deployed environment.
Packaging	Docker, Docker Compose	One image per service; network isolation expressed declaratively.
GPU	NVIDIA CUDA via WSL2 passthrough	Verified on an RTX 4060 (8 GB) before any weights were wired in.
Testing	pytest, compose smoke test	139 unit/integration tests plus a live-stack script that asserts against real Postgres and object storage.
Supply chain	gitleaks	Full-history secrets scan gates every push to the public remote.

Why not Kubernetes yet

Cluster provisioning, secrets, ingress and persistent volumes are multi-day work on their own, and on a single GPU node they buy nothing the compose stack does not already do. Compose runs the identical service decomposition, so the migration stays a packaging change rather than a redesign. The trigger is a second node, a real availability requirement, or autoscaling the GPU pod — that last one being the cost driver. Now that the delivery deadline has been lifted, this is schedulable rather than deferred; available time is not by itself the reason to do it.

6. Implementation Plan

Hardening is deferred in explicit tiers. The point of naming tiers is that nothing gets skipped silently — a deferred item is written down as deferred, not quietly omitted.

The phases below are the delivery record of the original five-day build. That deadline has since been lifted, so the tiers now describe dependency and order rather than what fits in a week. Some deferrals were justified purely by the clock and are now open; others were gated on something real — trained weights, a held-out calibration set, legal sign-off — and are unchanged by having more time. Available time schedules work; it does not by itself justify building something half-way.

PHASE	DELIVERABLE	CONTENTS
Day 1	Skeleton & contracts	Schema with the hold flag, config, storage abstraction, queue and DLQ semantics, rate limiter.
Day 2	Ingest & isolation	Presigned upload, gateway endpoints, WebSocket status. CPU preprocessing worker <i>with</i> its container lockdown — isolation ships with the parser, not after it.
Day 3	Scoring & routing	Inference workers behind a swappable backend, aggregation, worst-case face rollup, band routing, review flags.
Day 4	Retention	TTL deletion, hold gate across all three delete triggers, extended retention window, crash-recovery sweeper.
Day 5	End-to-end proof	Compose stack running end-to-end, smoke test asserting against real Postgres and buckets, GPU passthrough verified.

6.1 Current status

- ✓ Compose stack runs end-to-end; smoke test passes every check
- ✓ 139 automated tests passing
- ✓ Deletion verified against the storage backend — objects confirmed absent, not merely flagged
- ✓ Hold gate tested across all three delete triggers, held and unheld
- ✓ Every status the schema permits has a sweep path, checked against the SQL that runs rather than a restatement of it
- ✓ Failed jobs report why, on both the WebSocket and the polling view
- ✓ Audit trail asserted by reading Postgres columns back, not trusting the API response
- ✓ Upload size cap proven enforced by object storage (oversized upload rejected)
- ✓ GPU passthrough verified end-to-end before wiring the model backend
- Trained weights not yet wired — the stub backend is active by default

6.2 Next

- Wire the PyTorch backend and load real face and audio weights; run the launch calibration pass separately for each model.
- Replace the review flag with a reviewer-facing surface once there is a reviewer.
- Redis Streams consumer groups, isotonic calibration, per-retrain recalibration.
- Adversarial-input pre-classifier — the largest known gap, deliberately not half-built.

6.3 Stated limitations

What this system does not claim

- **No adversarial robustness.** Scores are manipulable by adversarial perturbation of the input. This is documented, not hidden, and every result says so.
- **No legal hold.** The extended retention window is a fixed timer that auto-expires.
- **No production-validated calibration.** Calibration is a launch snapshot, not tied to a recalibration pipeline.
- **No compliance assertion.** Scoped deletion and a partial audit trail are meaningfully better than nothing, but compliance is a legal determination this codebase does not get to make.
- **Not a detector yet.** The default backend is a deterministic stub that derives a score from a hash of the input. It exists so the pipeline could be built and proven before weights landed. It carries no detection meaning, and says so in every result it produces.

The last point is the one most worth stating plainly: what has been proven so far is that the *system* works end to end — ingest, scoring, aggregation, routing, deletion and audit. What has not been proven is *detection accuracy*, which is a property of weights that are not yet loaded. Conflating the two would be the single most misleading thing this document could do.